

1) Axios client trung tâm với baseURL dùng chung + interceptor xử lý lỗi thống nhất

```
const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
})
// -----
// Xử lý lỗi tập trung cho toàn bộ API module
// -----
export interface ApiError {
  message: string
  status?: number
}

export function handleApiError(error: unknown): ApiError {
  if (axios.isAxiosError(error)) {
    return {
      message:
        (error.response?.data as any)?.detail ||
        (error.response?.data as any)?.message ||
        error.message ||
        'Đã có lỗi xảy ra, vui lòng thử lại.',
      status: error.response?.status,
    }
  }
  if (error instanceof Error) {
    return { message: error.message }
  }
  return { message: 'Lỗi không xác định' }
}

// Interceptor: mọi lỗi đi qua apiClient đều được chuẩn hóa thành ApiError
apiClient.interceptors.response.use(
  (response) => response,
  (error) => Promise.reject(handleApiError(error))
)
```

2) Module usersApi sẵn sàng cho các tính năng người dùng trong tương lai

```
export const usersApi = {
  async getProfile(): Promise<UserProfile> {
    const res = await apiClient.get<BackendUser>('/users/me')
    return mapBackendUserToProfile(res.data)
  },

  async updateProfile(payload: UpdateProfilePayload): Promise<UserProfile> {
    const res = await apiClient.patch<BackendUser>('/users/me', payload)
    return mapBackendUserToProfile(res.data)
  },

  // TODO: backend chưa có endpoint /users/me/tickets,
  // bổ sung khi có API vé của tôi
  // async getMyTickets(): Promise<UserTicket[]> {
  //   const res = await apiClient.get<UserTicket[]>('/users/me/tickets')
  //   return res.data
  // },
}
```

3) Product store gọi service bằng async/await, không gọi axios trực tiếp /movies.ts

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import { movieService, type Movie, type Showtime } from '~/services/api'

export const useMoviesStore = defineStore('movies', () => {
  const movies = ref<Movie[]>([])
  const recommendations = ref<Movie[]>([])
  const loading = ref(false)
  const activeMovie = ref<Movie | null>(null)
  const activeShowtimes = ref<Showtime[]>([])

  async function fetchMovies() {
    loading.value = true
    try {
      movies.value = await movieService.getAll()
    } catch (e) {
      console.error('Failed to load movies:', e)
    } finally {
      loading.value = false
    }
  }
})
```

```

async function fetchRecommendations() {
  try {
    recommendations.value = await movieService.getRecommendations()
  } catch (e) {
    console.error('Failed to load recommendations:', e)
  }
}

async function fetchMovieDetails(id: string) {
  loading.value = true
  try {
    activeMovie.value = await movieService.getById(id)
    activeShowtimes.value = await movieService.getShowtimes(id)
  } catch (e) {
    console.error(`Failed to load details for movie ${id}:`, e)
    activeMovie.value = null
    activeShowtimes.value = []
  } finally {
    loading.value = false
  }
}

async function searchMovies(query: string) {
  if (!query.trim()) {
    await fetchMovies()
    return
  }
  loading.value = true
  try {
    movies.value = await movieService.searchSemantically(query)
  } catch (e) {
    console.error('Failed search:', e)
  } finally {
    loading.value = false
  }
}

return {
  movies,
  recommendations,
  loading,
  activeMovie,
  activeShowtimes,
  fetchMovies,
  fetchRecommendations,

```

```

    fetchMovieDetails,
    searchMovies
  }
})

```

- 4) Trang danh sách/chi tiết phim không import axios trực tiếp, pages/index.vue

```

onMounted(async () => {
  await moviesStore.fetchMovies()
})

```

- 5) Build thành công sau khi thêm usersApi và error handling bằng lệnh npm run build

```

PS C:\SE Special Project 2\PPQ\frontend> npm run build
>>

> cineai-frontend@1.0.0 build
> nuxt build

  Building Nuxt for production...
  • Nuxt 3.21.8 (with Nitro 2.13.4, Vite 7.3.6 and Vue 3.5.35)
  i Using Tailwind CSS from ~/assets/css/tailwind.css
  |
  • Nitro preset: node-server
  i Building client...
  i vite v7.3.6 building client environment for production...
  i ✓ 280 modules transformed.
  i .nuxt/dist/client/manifest.json 6.82 kB | gzip: 0.97 kB

  [
    .output/server/index.mjs (1.67 kB) (737 B gzip)
    .output/server/package.json (1.55 kB) (609 B gzip)
  ]
  Σ Total size: 2.98 MB (750 kB gzip)
  ✓ You can preview this build using node .output/server/index.mjs
  |
  | ✨ Build complete!
○ PS C:\SE Special Project 2\PPQ\frontend>

```

- 6) Request movies trả về status 500 nghĩa là frontend đã gọi đúng qua apiClient, nhưng backend đang bị lỗi khi xử lý — đây là lỗi phía server,

```
// Set this to false when backend (FastAPI) is running
const USE MOCK = false
const API_BASE_URL = import.meta.env.NUXT_PUBLIC_API_BASE || 'http://localhost:8000/api/v1'
```

The screenshot displays a web browser window with the URL `localhost:3000`. The browser's developer tools are open, showing the Network tab. The network traffic is filtered to show only 'Fetch/XHR' requests. A table of network requests is visible, with columns for Name, Status, Type, Time, and Fulfilled by. The table shows two requests: 'movies' (Status: 500, Type: xhr, Time: 0 B, Fulfilled by: api.ts:583) and 'dev.json' (Status: 304, Type: fetch, Time: 287 B, Fulfilled by: index.mjs:21). A tooltip is visible over the 'api.ts:583' link, showing the source code location: `(anonymous) @ api.ts:583`, `(anonymous) @ movies.ts:15`, and `(anonymous) @ index.vue:11`. The browser's console shows 2 / 259 requests, 0.3 kB / 4,112 kB transferred, and 0.1 kB / 11,962 kB resources. The page content shows a movie website with the title 'Phim đang chiếu' and a description 'Công nghệ dẫn đầu trải nghiệm'.

Name	Status	Type	Time	Fulfilled by
movies	500	xhr	0 B	api.ts:583
dev.json	304	fetch	287 B	index.mjs:21